

DNA Sequence Matching using Parallel Computing CPU vs GPU Performance Analysis

A Course Project Report
Submitted in partial fulfillment of the completion of the course
23CS302PC303 – High Performance Computing

In III Year II Semester B.Tech – CSE

Submitted by
Gandra Rakesh Rao
Hall Ticket Number: 2303A51851
Under the supervision of
Dr. Sharan Bediga
Assistant Professor, SOCS & AI

Academic Year 2025–2026



Department of Computer Science & Artificial Intelligence
SR University
Warangal, Telangana, India

INDEX

S.No	Section
1	Introduction
2	Problem Statement
3	Methodology / Procedure
4	Code
5	Output Screenshots
6	Observations, Limitations and Future Scope
7	References

1. Introduction

DNA sequence matching is an important technique used in bioinformatics and forensic science to identify individuals based on their genetic information. In real-world scenarios such as crime investigations, DNA samples collected from a crime scene are compared with a large database of individuals to find a match.

However, as the size of the dataset increases, the time required for matching also increases significantly. Traditional serial computation becomes slow and inefficient for large datasets. To overcome this limitation, parallel computing techniques such as OpenMP (for CPU parallelization) and CUDA (for GPU acceleration) are used.

This project focuses on comparing the performance of serial CPU, parallel CPU, and GPU implementations for DNA sequence matching.

DNA Sample Collection and Dataset Representation

In this project, DNA samples are represented in a simplified digital format using sequences of characters such as A, T, C, and G, which correspond to the four nucleotide bases found in real DNA.

In real-world scenarios, DNA samples are collected from biological evidence such as hair, blood, or skin cells at a crime scene. After collection, the DNA is processed in laboratories to extract the genetic sequence. This sequence is then converted into a digital format and stored in databases for further analysis and comparison.

In large-scale systems, these databases may contain thousands or even millions of DNA records.

Dataset Used in This Project

For this project, instead of using real biological data, synthetic DNA sequences are generated using a Python program. The dataset is stored in a file named `Human.txt`, where each line represents an individual along with their DNA sequence.

Each record in the dataset contains:

- A unique identifier (e.g., Person1, Person2, etc.)
- A randomly generated DNA sequence of fixed length

Example format of the dataset:

```
Person1 ATCGTACGTTAGC...
Person2 TCGATCGTAGCTA...
Person3 GTCGTACGATCGA...
```

During program execution, the DNA obtained from the crime scene (target DNA) is provided as input by the user. The system then compares this target DNA with all stored DNA sequences to identify a match.

This dataset simulates real-world DNA databases and allows the implementation of efficient matching algorithms using serial, parallel, and GPU-based approaches.

2. Problem Statement

The main objective of this project is to efficiently match a given DNA sequence (crime DNA) with a large dataset of DNA sequences stored in a file.

The problem involves the following tasks:

- Reading DNA data of multiple individuals from a dataset file
- Comparing each DNA sequence with the target DNA
- Identifying whether a matching DNA sequence exists
- Measuring execution time for different computational approaches

Challenges in the Problem

One of the major challenges in this problem is the large size of the dataset. In real-world scenarios, DNA databases may contain thousands or even millions of records. Performing comparisons sequentially (one by one) becomes computationally expensive and time-consuming.

The time complexity of DNA matching can be expressed as:

$$O(n \times m)$$

where:

- n = number of DNA samples
- m = length of each DNA sequence

As the dataset size increases, the total execution time grows significantly, making serial processing inefficient.

Need for High Performance Computing

To overcome these limitations, High Performance Computing (HPC) techniques are required. Parallel computing allows multiple DNA sequences to be processed simultaneously, thereby reducing execution time.

This project explores three different approaches:

- **Serial CPU Execution:** Processes DNA sequences one at a time
- **Parallel CPU Execution (OpenMP):** Utilizes multiple CPU cores for concurrent processing
- **GPU Execution (CUDA):** Uses thousands of threads to perform massive parallel comparisons

Objective of the Project

The primary objective is not only to identify matching DNA sequences but also to analyze and compare the performance of different computational techniques.

The project aims to:

- Reduce execution time for large datasets
- Improve scalability using parallel processing
- Demonstrate the efficiency of GPU computing over CPU-based approaches

Real-World Relevance

Efficient DNA matching is critical in real-world applications such as:

- Criminal investigations and forensic analysis
- Medical and genetic research
- Ancestry and genealogy studies

Faster processing enables quicker decision-making, which is especially important in time-sensitive scenarios like crime investigations.

3. Methodology / Procedure

The methodology of this project focuses on efficient DNA sequence matching using High Performance Computing techniques. It integrates dataset generation, sequential processing, parallel CPU execution, and GPU-based acceleration.

1. Dataset Preparation

The DNA dataset is generated using a Python program and stored in a file named `Human.txt`. Each record represents an individual along with their DNA sequence.

Each DNA sequence consists of characters A, T, C, and G, with a fixed length. The dataset simulates real-world DNA databases containing thousands of records.

2. DNA Matching Logic

The core operation of this project is comparing a target DNA sequence (crime DNA) with each DNA sequence in the dataset.

The comparison is performed character by character:

$$match = \{ 1, \text{if all characters match} 0, \text{otherwise} \}$$

This process is repeated for all DNA samples.

3. Serial CPU Implementation

In the serial approach, each DNA sequence is compared one by one using a loop.

- Simple and straightforward implementation
- High execution time for large datasets
- No parallelism is utilized

4. Parallel CPU Implementation (OpenMP)

To improve performance, OpenMP is used to parallelize the comparison process.

- Multiple CPU cores execute comparisons simultaneously
- Each thread processes a subset of DNA sequences
- Reduces execution time compared to serial approach

5. CUDA GPU Parallelization

GPU acceleration is used to achieve maximum performance by exploiting massive parallelism.

Thread Mapping

Each GPU thread is assigned to compare one DNA sequence with the target DNA.

If n is the number of DNA samples:

$$TotalThreads = n$$

$$Blocks = \left\lceil \frac{n}{256} \right\rceil$$

Kernel Execution

The CUDA kernel `matchDNA()` performs the following operations:

- Each thread reads one DNA sequence
- Compares it with the target DNA
- Stores the result (match or no match) in an output array

Memory Management

- DNA data is flattened into a 1D array for efficient GPU access
- Memory is allocated on the device using `cudaMalloc()`
- Data is transferred using `cudaMemcpy()`
- Results are copied back to the host after execution

6. Performance Measurement

Execution time is measured for all implementations:

- Serial CPU using `clock()`
- Parallel CPU using `omp_get_wtime()`
- GPU using CUDA events

This allows comparison of performance across different approaches.

7. Complexity Analysis

The time complexity of DNA matching is:

$$O(n \times m)$$

where:

- n = number of DNA samples
- m = length of DNA sequence

Parallel computing reduces execution time by distributing work across multiple processing units.

8. Overall Workflow

1. Generate DNA dataset
2. Read DNA data from file
3. Input crime DNA
4. Perform matching using:
 - Serial CPU
 - Parallel CPU (OpenMP)
 - GPU (CUDA)
5. Measure execution time
6. Compare performance

4. Code

4.1 Dataset Generation

```
1 %%writefile generate_dataset.py
2 import random
3
4 dna_chars = ['A', 'T', 'C', 'G']
5
6 num_people = 50000
7 dna_length = 800
8
9 with open("Human.txt", "w") as f:
10     for i in range(1, num_people + 1):
11         dna = ''.join(random.choice(dna_chars) for _ in range(
12             dna_length))
13         f.write(f"Person{i} {dna}\n")
14
15 print("Dataset generated")
16
17 !python generate_dataset.py
18
19 from google.colab import files
20 files.download("Human.txt")
```

4.2 CPU CODE (Serial vs Parallel)

```
1 %%writefile cpu_dna.c
2 #include <stdio.h>
3 #include <string.h>
4 #include <time.h>
5 #include <omp.h>
6
7 #define MAX 50000
8 #define LEN 1000
9
10 char names[MAX][50];
11 char dna[MAX][LEN];
12 char crime_dna[LEN];
13
14 int compareDNA(char *a, char *b) {
15     for (int i = 0; a[i] != '\0'; i++) {
```

```

16         if (a[i] != b[i]) return 0;
17     }
18     return 1;
19 }
20
21 int main() {
22     FILE *fp = fopen("Human.txt", "r");
23     int n = 0;
24
25     while (fscanf(fp, "%s %s", names[n], dna[n]) != EOF) n++;
26     fclose(fp);
27
28     printf("Total Records: %d\n", n);
29
30     // USER INPUT
31     printf("Enter Crime DNA: ");
32     scanf("%s", crime_dna);
33
34     int repeat;
35     printf("Enter repeat count: ");
36     scanf("%d", &repeat);
37
38     // SERIAL
39     clock_t s1 = clock();
40     for (int r = 0; r < repeat; r++)
41         for (int i = 0; i < n; i++)
42             comparedDNA(dna[i], crime_dna);
43     clock_t e1 = clock();
44
45     double serial = (double)(e1 - s1) / CLOCKS_PER_SEC;
46
47     // PARALLEL
48     double s2 = omp_get_wtime();
49     for (int r = 0; r < repeat; r++) {
50         #pragma omp parallel for schedule(static)
51         for (int i = 0; i < n; i++)
52             comparedDNA(dna[i], crime_dna);
53     }
54     double e2 = omp_get_wtime();
55
56     double parallel = e2 - s2;
57
58     printf("Serial = %f sec\n", serial);
59     printf("Parallel = %f sec\n", parallel);
60
61     // SAVE
62     FILE *out = fopen("cpu_output.txt", "w");
63     fprintf(out, "%f %f", serial, parallel);
64     fclose(out);
65
66     return 0;

```

```

67 }
68 #
69
70 To Run the Code Command :
71 !gcc -fopenmp cpu_dna.c -o cpu
72
73 print("Enter the repeat count as the 'how many times loop runs'")
74 print("Beter to give the 10000 ten thousand times iteratons ")
75 !./cpu

```

4.3 GPU CODE (CUDA)

```

1  %%writefile gpu_dna.cu
2  #include <stdio.h>
3  #include <string.h>
4  #include <stdlib.h>
5  #include <cuda.h>
6
7  #define MAX 50000
8  #define LEN 1000
9
10 __global__ void matchDNA(char *dna, char *crime, int *res, int n,
    int len) {
11     int i = threadIdx.x + blockIdx.x * blockDim.x;
12     if (i >= n) return;
13
14     int match = 1;
15     for (int j = 0; j < len; j++) {
16         if (dna[i * len + j] != crime[j]) {
17             match = 0;
18             break;
19         }
20     }
21     res[i] = match;
22 }
23
24 int main() {
25     FILE *fp = fopen("Human.txt", "r");
26
27     static char dna[MAX][LEN];
28     char crime[LEN];
29
30     int n = 0;
31     while (fscanf(fp, "%s %s", dna[n]) != EOF) n++;
32     fclose(fp);
33
34     printf("Total Records: %d\n", n);
35
36     // USER INPUT
37     printf("Enter Crime DNA: ");

```

```

38     scanf("%s", crime);
39
40     int len = strlen(crime);
41
42     char *flat = (char*)malloc(n * LEN);
43     for (int i = 0; i < n; i++)
44         memcpy(&flat[i*LEN], dna[i], LEN);
45
46     char *d_dna, *d_crime;
47     int *d_res;
48
49     cudaMalloc(&d_dna, n*LEN);
50     cudaMalloc(&d_crime, LEN);
51     cudaMalloc(&d_res, n*sizeof(int));
52
53     cudaMemcpy(d_dna, flat, n*LEN, cudaMemcpyHostToDevice);
54     cudaMemcpy(d_crime, crime, LEN, cudaMemcpyHostToDevice);
55
56     int threads = 256;
57     int blocks = (n + threads - 1)/threads;
58
59     cudaEvent_t s, e;
60     cudaEventCreate(&s);
61     cudaEventCreate(&e);
62
63     cudaEventRecord(s);
64
65     matchDNA<<<blocks, threads>>>(d_dna, d_crime, d_res, n, len);
66     cudaDeviceSynchronize();
67
68     cudaEventRecord(e);
69     cudaEventSynchronize(e);
70
71     float ms;
72     cudaEventElapsedTime(&ms, s, e);
73
74     printf("GPU = %f ms\n", ms);
75
76     FILE *out = fopen("gpu_output.txt", "w");
77     fprintf(out, "%f", ms/1000.0);
78     fclose(out);
79
80     return 0;
81 }
82
83 To Run the Code Command :
84 !nvcc gpu_dna.cu -o gpu
85 !./gpu

```

4.4 Python Visualization Code

```
1
2 import matplotlib.pyplot as plt
3
4 with open("cpu_output.txt") as f:
5     serial, parallel = map(float, f.read().split())
6
7 with open("gpu_output.txt") as f:
8     gpu = float(f.read())
9
10 print("\nFINAL RESULT")
11 print("Serial    :", serial)
12 print("Parallel  :", parallel)
13 print("GPU       :", gpu)
14
15 methods = ['Serial', 'Parallel', 'GPU']
16 times = [serial, parallel, gpu]
17
18 plt.figure()
19 plt.bar(methods, times)
20
21 for i, v in enumerate(times):
22     plt.text(i, v, f"{v:.4f}", ha='center')
23
24 plt.title("Performance Comparison")
25 plt.ylabel("Time (sec)")
26 plt.show()
```

5. Output Screenshots

CPU Results

```
Enter the repeat count as the 'how many times loop runs'  
Beter to give the 10000 ten thousand times iteratons  
Total Records: 50000
```

```
Enter Crime DNA:
```

```
GTAGGCCTCCTTCTTCCAGCGGACACGCGTTAGGTGTTCTGGTGTAACGTATAAGGCTCTACACAAA  
AGACGGTAAAGTATTTGCCAGACTCACGATGACAAAGCCAGAGGTGCGATGGACCTTGGGACTGGT  
CCTTAACCTAGGCTGCTTTAACATTTTCGACCTCCATTATCAACTTTCTGCCCCCCTGTGAAAGTG  
AATAGCGACGGTTTAAAGCTGTAGCCTGCCGGTGCAGCGAACCCTGGCAAGAATAGGTTTCCATATT  
TTTTGCTTAAGAGGAGATAGCACTCCCAGCTGGTGGTGCCGATCATCTGTGTGGCCACCCGTAACC  
TGTAGGTAATGAGGCGATCTCGATATAATCGTGTATTAGTAGCCGTCTATGAAGTTGCGACAGAAC  
CCGGGTGATGCAAGTACCCTAATGCCCTATGCGGTATAATTTAGACCCAGGTAATGTTCTCCTGCA  
CCGGCCGAAGGCGGGGAACCTGGTAGCGGCTAGCTAATCATAACCCACTGAGAGGCTAGGCCATGCA  
GAAAAGGGGGACCCAATCTCCGACGGTCCCAGTTGACAGTGCGCGTATAATGTGCCGTATCCCCG  
TTCGTATCCTCCAAGCCACTTTGTATTAGTCTACTGTGGTTTAGCCGAGAGTCCATGGGAGATCTA  
CGCATTGAACGGCATCCACTAAAGACGGAGTGCGCATTGGAGAAATGAGGGCGAAGAGATGTACTA  
TGTTGAAAGATCCTTCGCGCAGTTGCCCATCAGATTCCAACCGCATGGCAACCCCGAGGGTGTCGA  
TTGTCGGGT
```

```
Enter repeat count: 10000  
1000
```

```
Serial    = 8.025631 sec  
Parallel  = 5.256038 sec
```

Figure 5.1: Benchmark Results for Serial, Parallel CPU Implementations

GPU Results

```
nvcc warning : Support for offline compilation for architectures prior
'<compute/sm/lto>_75' will be removed in a future release
(Use -Wno-deprecated-gpu-targets to suppress warning).
```

Total Records: 50000

Enter Crime DNA:

```
GTAGGCCTCCTTCTTCCAGCGGACACGCGTTAGGTGTTCTGGTGTAACGTATAAGGCTCTACACAAA
AGACGGTAAAGTATTTGCCAGACTCACGATGACAAAGCCAGAGGTGCGATGGACCTTGGGACTGGT
CCTTAACCTAGGCTGCTTTAACATTTTCGACCTCCATTATCAACTTTCTGCCCCCTGTGAAAGTG
AATAGCGACGGTTTAAAGCTGTAGCCTGCCGGTGCAGCGAACCCCTGGCAAGAATAGGTTTCCATATT
TTTTGCTTAAGAGGAGATAGCACTCCCAGCTGGTGGTGGCGATCATCTGTGTGGCCACCCGTAACC
TGTAAGTAATGAGGCGATCTCGATATAATCGTGTATTAGTAGCCGTCTATGAAGTTGCGACAGAAC
CCGGGTGATGCAAGTACCCTAATGCCCTATGCGGTATAATTTAGACCCAGGTAATGTTCTCCTGCA
CCGGCCGAAGGCGGGGAAGTGGTAGCGGCTAGCTAATCATAACCCACTGAGAGGCTAGGCCATGCA
GAAAAGGGGGACCCAATCTCCGACGGTCCCAGATTGACAGTGCGCGTATAATGTGCCGTATCCCCG
TTCGTATCCTCCAAGCCACTTTGTATTAGTCTACTGTGGTTTAGCCGAGAGTCCATGGGAGATCTA
CGCATTGAACGGCATCCACTAAAGACGGAGTGCGCATTGGAGAAATGAGGGCGAAGAGATGTACTA
TGTTGAAAGATCCTTCGCGCAGTTGCCCATCAGATTCCAACCGCATGGCAACCCCGAGGGTGTCGA
TTGTCGGGT
```

GPU = 105.361504 ms

Figure 5.2: GPU Execution Output for DNA Sequence Matching

FINAL RESULT

Method	Execution Time (seconds)
Serial	8.025631
Parallel	5.256038
GPU	0.105362

Comparison graph

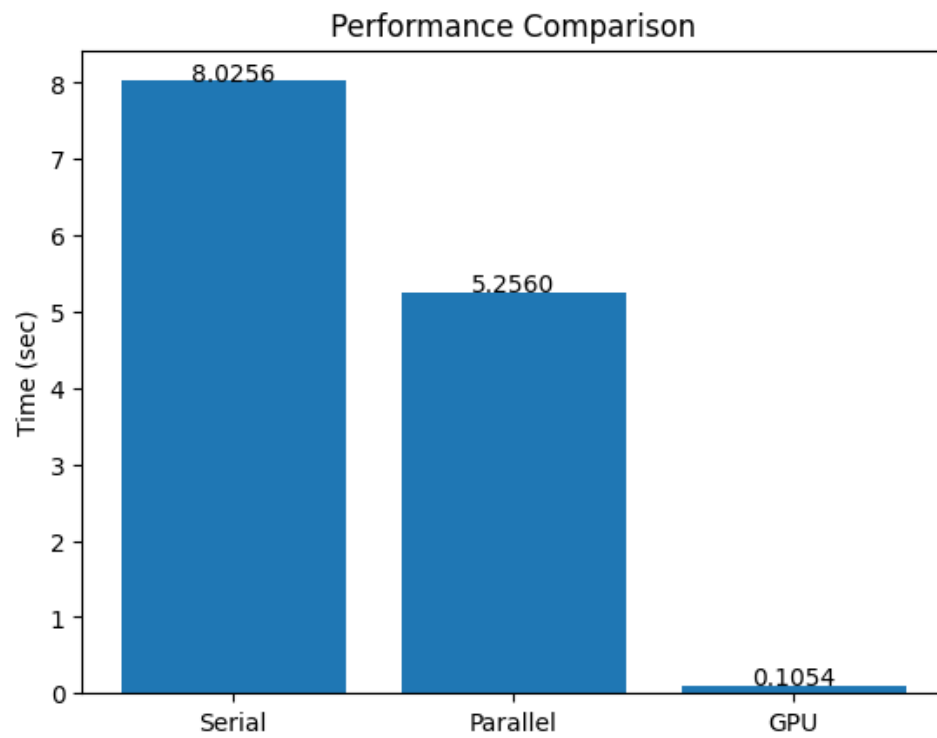


Figure 5.3: Performance Comparison of Serial CPU, Parallel CPU, and GPU Implementations

6. Observations, Limitations and Future Scope

Observations

- GPU acceleration provides significant speedup compared to both serial and parallel CPU implementations.
- Parallel CPU (OpenMP) improves performance over serial execution but is limited by the number of available cores.
- CUDA-based GPU implementation efficiently utilizes thousands of threads to perform DNA comparisons simultaneously.
- Execution time decreases drastically as parallelism increases, especially for large datasets.
- The performance comparison clearly demonstrates the advantage of GPU computing for data-intensive tasks like DNA sequence matching.

Limitations

- The current implementation performs exact matching only and does not handle partial or approximate matches.
- GPU memory limits the size of the dataset that can be processed at once.
- Data transfer between CPU (host) and GPU (device) introduces additional overhead.
- The program assumes fixed-length DNA sequences, which may not always be realistic.
- Performance gains depend on the availability of suitable GPU hardware.

Future Scope

- Implement approximate DNA matching algorithms such as sequence alignment (e.g., Needleman-Wunsch or Smith-Waterman).
- Optimize CUDA implementation using shared memory and memory coalescing techniques.
- Extend the system to handle variable-length DNA sequences.
- Integrate the solution with real-world DNA databases for practical applications.
- Expand the implementation to distributed systems or multi-GPU environments for handling very large datasets.

Conclusion

This project successfully demonstrates the application of High Performance Computing techniques for DNA sequence matching. The comparison between serial CPU, parallel CPU, and GPU-based implementations shows that parallel computing significantly reduces execution time.

Among all approaches, the GPU implementation provides the highest performance due to its massive parallel processing capability. This makes it highly suitable for large-scale DNA analysis tasks.

Overall, the project highlights how computational efficiency can be improved using parallelism, making it possible to process large datasets quickly and effectively. Such techniques are highly valuable in real-world applications like forensic analysis, medical research, and genetic studies.

7. References

- About DNA – Wikipedia: <https://en.wikipedia.org/wiki/DNA>
- CUDA Samples (Parallel Computing Examples) : <https://github.com/NVIDIA/cuda-samples>
- OpenMP Examples (Parallel CPU Programming) : <https://github.com/OpenMP/Examples>
- Research Papers (Google Scholar / IEEE / Journals) : <https://www.nature.com/articles/nature24286>
- Barnes–Hut Algorithm – NASA/ADS: <https://ui.adsabs.harvard.edu/abs/1986Natur.324..446B>
- Parallel Programming with CUDA – NVIDIA Developer Blog: <https://developer.nvidia.com/blog/tag/cuda/>
- Graphs Performance from Google Scholar Research Paper's: <https://www.sciencedirect.com/science/article/pii/S030439750600017X>
- Google Colab: <https://colab.research.google.com/drive/1OZ3hf02JuxOu4EohNOusp=sharing>